



ESTRUTURAS DE DADOS E ALGORITMOS

Algoritmos de Força Bruta

Aula 3 · experimentar tudo, com método

Prof. Doutor Luís Simões da Cunha
luiscunha@ismt.pt

Três perguntas para aquecer

1

Dois ciclos for encadeados, cada um de 0 a n. Qual a complexidade?

2

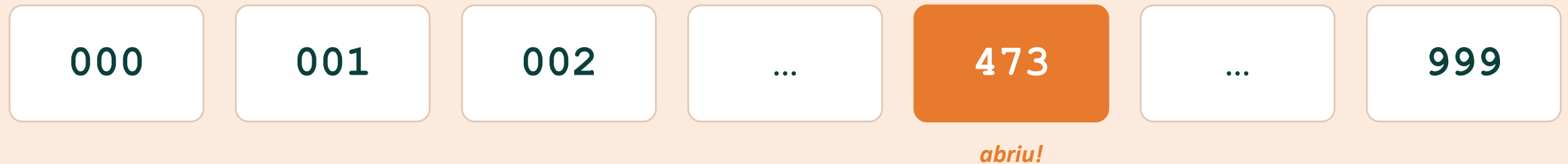
Quantas permutações diferentes existem de N elementos?

3

Porque é que analisamos sempre o pior caso?

Responda sem olhar para os apontamentos: recordar é o que consolida a memória.

Esqueceu-se do código do cadeado de 3 dígitos. Como o abre?



1 000

tentativas no máximo (3 dígitos)

10 000

com 4 dígitos

Garantido

se o código existe, encontra-o

Isto é força bruta: sem truques, só método e paciência — a do computador.

Força bruta: simples, garantida e cara

1

O que é

Definição, exemplos e características.

2

Quando usar

Cinco razões para não a desprezar.

3

Ordenar à força

Selection Sort e Bubble Sort, $O(n^2)$.

4

Pesquisa exaustiva

O problema do caixeiro-viajante.

Pergunta-guia: quando é que "experimentar tudo" é uma boa ideia — e quando é impossível?

1

PARTE 1

O que é a força bruta

A abordagem mais direta possível.

Resolver o problema "à letra"

A força bruta é uma abordagem **direta**, baseada no **enunciado do problema** e nas **definições** dos conceitos envolvidos.

1

Experimenta tudo

Tenta as soluções possíveis até encontrar a correta: procura exaustiva.

2

Sem atalhos

Não usa otimizações nem heurísticas inteligentes.

3

A força é da máquina

A "força" é o poder de cálculo do computador, não o engenho de quem programa.

Quatro algoritmos de força bruta. Qual a complexidade?

- A

 Procurar um elemento num array não ordenado $O(?)$
- B

 Verificar se um texto contém uma palavra $O(?)$
- C

 Verificar se uma palavra é um palíndromo (ex.: "radar") $O(?)$
- D

 Encontrar todos os divisores de um número n $O(?)$

Pense primeiro: o que é que o algoritmo tem de percorrer?

Qual a complexidade? As respostas

A

Procurar num array não ordenado **$O(n)$**

Pode ter de ver todos os elementos.

B

Texto contém uma palavra **$O(n \cdot m)$** Em cada posição do texto (n), compara a palavra (m).

C

Palíndromo **$O(n)$** Compara as pontas, a caminhar para o meio: $n/2$ passos.

D

Divisores de n **$O(n)$** Testa 1, 2, ..., n . (Até $\sqrt{n}$ chega — porquê?)

Nenhum é exponencial: a força bruta nem sempre é lenta. Tudo depende do tamanho do espaço de procura.

Duas vantagens e dois custos

COMO FUNCIONA

Procura exaustiva

Explora todas as opções do espaço de procura, sem estratégia para reduzir o número de verificações.

VANTAGEM

Simplicidade

Fácil de conceber e de implementar: é direta e sistemática, sem lógica complexa.

DESVANTAGEM

Custo elevado

Muitas vezes $O(2^n)$ ou $O(n!)$: impraticável para inputs grandes.

VANTAGEM

Solução garantida

Se existe solução, encontra-a. Útil quando a correção importa mais do que a eficiência.

2

PARTE 2

Quando usar

Raramente a mais inteligente, muitas vezes a mais útil.

Cinco razões para não desprezar a força bruta

1

Aplica-se a quase tudo

Ao contrário de outras estratégias, funciona numa enorme variedade de problemas.

2

Algoritmos razoáveis

Em ordenação, pesquisa, multiplicação de matrizes e texto dá soluções com valor prático.

3

Inputs pequenos

Se os dados são pequenos, um algoritmo sofisticado pode não compensar o esforço.

4

Instâncias pequenas

Mesmo ineficiente em geral, resolve bem casos pequenos de um problema difícil.

5

Ponto de referência

Serve de padrão para avaliar e testar alternativas mais eficientes.

"Primeiro, faça funcionar. Depois, faça rápido." A força bruta é muitas vezes o primeiro passo.

3

PARTE 3

Ordenar à força

Selection Sort e Bubble Sort.

Escolher o menor e pô-lo no lugar

Em cada passagem, percorre a parte por ordenar, encontra o menor e troca-o para a primeira posição livre.

início	64	25	12	22	11	
i = 0	11	25	12	22	64	mínimo 11 $\leftrightarrow$ 64
i = 1	11	12	25	22	64	mínimo 12 $\leftrightarrow$ 25
i = 2	11	12	22	25	64	mínimo 22 $\leftrightarrow$ 25
i = 3	11	12	22	25	64	25 já está no lugar

Truque para lembrar

Como escolher jogadores para uma equipa: escolhe-se sempre o "melhor" que resta.

Pseudocódigo e custo

```
SelectionSort(A[0..n-1])
para i ← 0 até n-2:
    min ← i
    para j ← i+1 até n-1:
        se A[j] < A[min]: min ← j
    trocar A[i] com A[min]
```

O ciclo interior (verde) é o que manda no custo.

Com 5 elementos: $4 + 3 + 2 + 1 = 10$ comparações = $5 \cdot 4 / 2$.

Comparações

$$(n-1) + (n-2) + \dots + 1$$

$$= n(n-1)/2 \rightarrow \Theta(n^2)$$

sempre — mesmo que o array já esteja ordenado

Trocas

$$\text{apenas } n - 1 \rightarrow O(n)$$

Útil quando mover dados é caro.

Os maiores "sobem" como bolhas

Compara vizinhos e troca-os se estiverem fora de ordem. Em cada passagem, o maior chega ao fim.

início	64	25	12	22	11	
passagem 1	25	12	22	11	64	4 trocas: o 64 sobe até ao fim
passagem 2	12	22	11	25	64	3 trocas: o 25 fica no lugar
passagem 3	12	11	22	25	64	1 troca: o 22 fica no lugar
passagem 4	11	12	22	25	64	1 troca: ordenado!

Laranja

o elemento que acabou de "subir" para a sua posição final nesta passagem.

Pseudocódigo e custo

```
BubbleSort(A[0..n-1])  
para i ← 0 até n-2:  
    para j ← 0 até n-2-i:  
        se A[j+1] < A[j]:  
            trocar A[j] com A[j+1]
```

Comparações

$$n(n-1)/2 \rightarrow \Theta(n^2)$$

tantas como no Selection Sort

Melhoria fácil

Se uma passagem inteira não fizer nenhuma troca, o array já está ordenado: pode parar. Com um array já ordenado, o custo passa a $O(n)$.

Trocas (pior caso)

$n(n-1)/2$ com o array ao contrário

Muito mais trocas do que o Selection Sort.

Selection Sort vs Bubble Sort

	Selection Sort	Bubble Sort
Ideia	Escolhe o menor e põe-no no lugar	Troca vizinhos fora de ordem
Comparações	$n(n-1)/2 \rightarrow \Theta(n^2)$	$n(n-1)/2 \rightarrow \Theta(n^2)$
Trocas (pior caso)	$n - 1$	$n(n-1)/2$
Array já ordenado	continua $\Theta(n^2)$	$O(n)$ com a melhoria
Complexidade	$O(n^2)$	$O(n^2)$

Na próxima aula: o Merge Sort (divisão e conquista) ordena em $O(n \log n)$ — muito mais depressa.

Quantas comparações faz o Selection Sort para ordenar...

10 000 elementos?

$$\approx 5 \times 10^7$$

comparações

a 10^9 por segundo: **0,05 s**

1 000 000 de elementos?

$$\approx 5 \times 10^{11}$$

comparações

a 10^9 por segundo: **≈ 8 min**

Multiplicar os dados por 100 multiplicou o tempo por 10 000. É a assinatura do $O(n^2)$.

4

PARTE 4

Pesquisa exaustiva

Força bruta para problemas combinatórios.

Gerar, filtrar, escolher

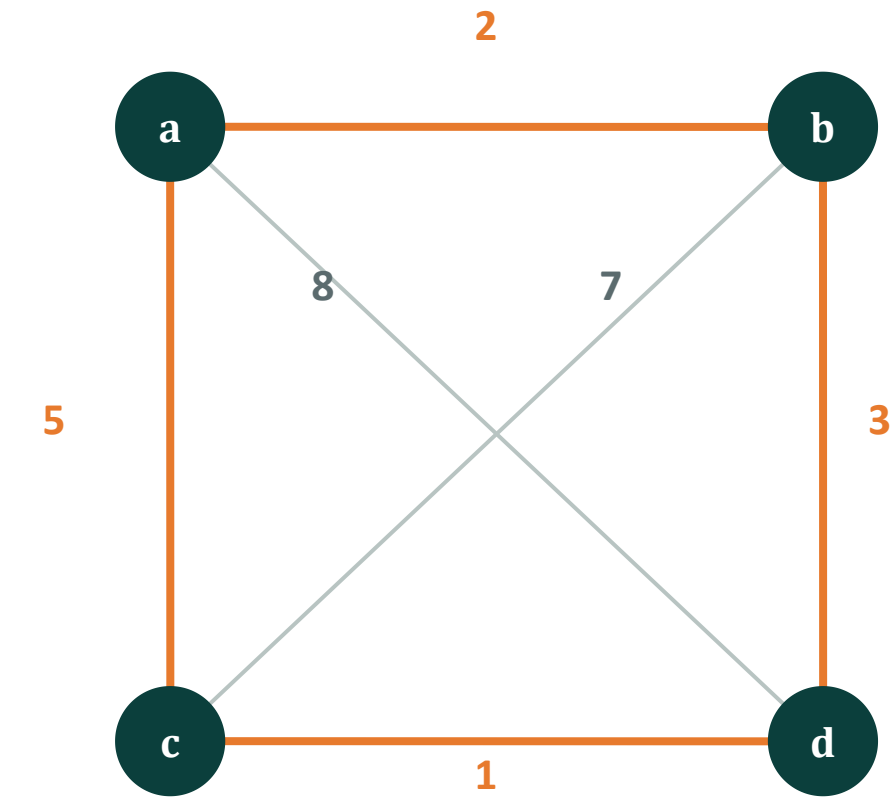
Pesquisa exaustiva = força bruta aplicada a problemas combinatórios (permutações, combinações, subconjuntos).



Atenção: a ideia é simples, mas implementá-la exige um algoritmo para gerar os objetos combinatórios (ex.: todas as permutações).

Qual é a viagem mais curta?

Visitar n cidades, cada uma exatamente uma vez, e voltar ao início, percorrendo a menor distância.



a laranja: uma viagem ótima (11)

Viagem	Distâncias	Total
$a \rightarrow b \rightarrow c \rightarrow d \rightarrow a$	$2 + 8 + 1 + 7$	18
$a \rightarrow b \rightarrow d \rightarrow c \rightarrow a$	$2 + 3 + 1 + 5$	11 ótima
$a \rightarrow c \rightarrow b \rightarrow d \rightarrow a$	$5 + 8 + 3 + 7$	23
$a \rightarrow c \rightarrow d \rightarrow b \rightarrow a$	$5 + 1 + 3 + 2$	11 ótima
$a \rightarrow d \rightarrow b \rightarrow c \rightarrow a$	$7 + 3 + 8 + 5$	23
$a \rightarrow d \rightarrow c \rightarrow b \rightarrow a$	$7 + 1 + 8 + 2$	18

Cada viagem aparece duas vezes (ida e o inverso): só há 3 viagens realmente diferentes.

Quantas viagens há para testar?

Com n cidades há $(n - 1)! / 2$ viagens diferentes. A 10^9 viagens por segundo:

$n = 4$

3

viagens

instantâneo

$n = 10$

181 440

viagens

0,2 ms

$n = 15$

$4,4 \times 10^{10}$

viagens

44 s

$n = 20$

$6,1 \times 10^{16}$

viagens

1,9 anos

$n = 25$

$3,1 \times 10^{23}$

viagens

9,8 milhões
de anos

Crescimento fatorial, $O(n!)$: a pesquisa exaustiva só serve para instâncias muito pequenas.

Agora é consigo



Selection Sort



Implemente em Java o Selection Sort, com o protótipo:

```
int[] selectionSort(int[] arr)
```

Teste com 64 25 12 22 11.



Bubble Sort



Implemente em Java o Bubble Sort, com o protótipo:

```
int[] bubbleSort(int[] arr)
```

Desafio: pare quando uma passagem não fizer trocas.



Caixeiro-viajante



Implemente em Java uma solução por pesquisa exaustiva para o problema do caixeiro-viajante.

Pista: guarde as distâncias numa matriz `int[][]` e gere todas as permutações das cidades. Teste com o exemplo (resposta: 11).

Três ideias a reter

1

Força bruta

Resolver o problema à letra, experimentando as possibilidades. Simples e com solução garantida.

2

Ordenar à força

Selection Sort e Bubble Sort fazem $n(n-1)/2$ comparações: $\Theta(n^2)$.

3

Pesquisa exaustiva

Gerar, filtrar, escolher. Correta, mas $O(n!)$ no caixeiro-viajante.

Próxima aula: dividir para reinar — algoritmos de divisão e conquista.

Bibliografia

1

Levitin, A. (2011). *Introduction to the Design and Analysis of Algorithms* (3.ª ed.). Pearson. Capítulo 3: Brute Force and Exhaustive Search.

Dúvidas? Prof. Doutor Luís Simões da Cunha · luiscunha@ismt.pt